

大数据时代 Redis 缓存的性能分析与优化

张蒙蒙 曹成茂

(安徽农业大学 工学院,安徽 合肥 230036)

摘要:随着云计算、物联网及大数据技术的发展,现代化的 Web 系统因业务数据量的庞大和高并发而在系统中引入了缓存技术,减少了对关系型数据库的直接访问所带来的服务器端的负载压力,提高数据库服务器的响应效率,降低浏览器的延迟。Redis 因其高性能分布式的内存缓存技术优势而被广泛的使用在现代 Web 系统中。本研究通过实验数据对 Redis 高性能原因进行分析,针对 Redis 使用中的一些方式,从不同的角度对 Redis 性能优化提出相应策略,以此更好的提升 Redis 的用户体验感。

关键词:大数据;Web;Redis;缓存技术;服务器

中图分类号:TP311

文献标识码:A

文章编号:1672-2868(2022)03-0080-08

Performance Analysis and Optimization of Redis Cache in the Era of Big Data

ZHANG Meng-meng CAO Cheng-mao

(School of Engineering, Anhui Agricultural University, Hefei Anhui 230036)

Abstract: With the development of cloud computing, Internet of things and big data technology, the cache technology has been introduced into the modern Web system because of the huge amount of data and high concurrency, which reduces the load pressure on the server end caused by the direct access to the relational database, improves the response efficiency of the database server and reduces the delay of the browser. Redis is widely used in modern Web systems because of its high-performance distributed memory cache technology. This paper will analyze the reasons for the high performance of Redis through the experimental data, and put forward corresponding strategies for Redis performance optimization from different angles based on experimental data, so as to better improve Redis user experience.

Key words: big data; Web; Redis; cache technology; server

0 引言

随着与计算机相关的物联网技术、云计算及大数据技术的发展扩大^[1],业务系统的数据量越来越庞大。对系统提出了更高的性能要求,传统的 Web 技术一般采用关系型数据库^[2],将数据存储到数据库中,一旦有数据请求,则从数据库服务器中获取数据。但是随着数据量的不断增加,用户对数据访问的相对集中,就会出现数据库服务器负担过重、数据库服务器的响应效率降低^[2]、浏览器的响应延迟增加等问题。针对以上问题,计算机可以通过硬件和软件技术的改进来匹配大数据量、高并发的需要。硬件的提升在一定范围内可以达到好的效果。但是更多的则是对软件技术的改进。在这种背景下,缓存技术应运而生。

从第一个缓存框架 Memcached 诞生以来,缓存技术一直广泛地存在于互联网应用中。而 Redis 也因其高性能分布式纯内存缓存技术的优势在现代互联网信息系统缓存中占据着绝对的优势。Redis 是一个基于键值对的存储结构系统,提供了许多实用的功能,如主从复制、RedisMQ 消息队列、Redis 集群等^[3]。Redis 缓存技术的应用使得对服务端的请求很多转移到缓存上从而大大缓解了服务器的负担^[4]。在 Redis 的使用中,可以通过许多方式来进行进一步提升 Redis 效率。将从 Redis 的性能分析入手,分析 Redis 的性能优势强大的原因,然后从使用中对 Redis 的性能提出一些改进的优化方案,进而更好的提升 Redis 的用户使用体验。如图 1 所示。

收稿日期:2021-11-23

作者简介:张蒙蒙(1990—),男,安徽合肥人,安徽农业大学工学院硕士研究生,主要从事计算机软件、农业物联网研究。

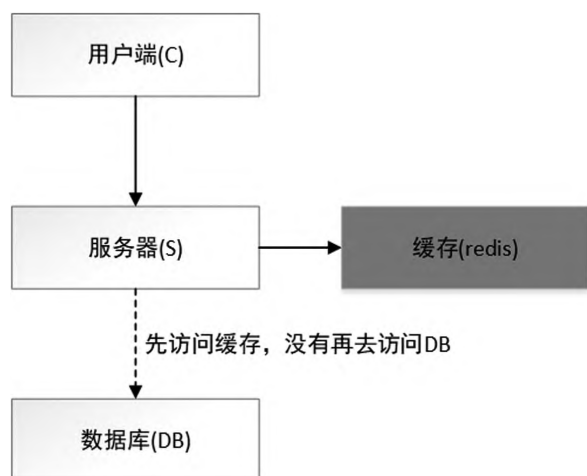


图1 加入 Redis 后的业务请求

1 Redis 的性能优势

Redis 官网中提到,在局域网环境下只要传输的包不超过一个 MTU(以太网下大约 1500 字节),那么对于 100 字节、1000 字节不同包大小的吞吐处理能力实际结果差不多。针对这种说法,从实验中进行验证发现和官网提的说法有一定的出入,这主要是因为网络及客户端的影响导致与理论的结果有偏差。在实验环境的测试下,可以从图 2 中看到,Redis 在处理 1000 字节的数据的时候,吞吐量都是稳定在 10 w/s 的请求量,当处理的数据不断增大的时候,吞吐量才慢慢开始下降,而实际的数据量很少能达到这个数量级。单个的 Redis 实例运行相对稳定,计算机单核 CPU 的有效利用率在 75%之间波动。

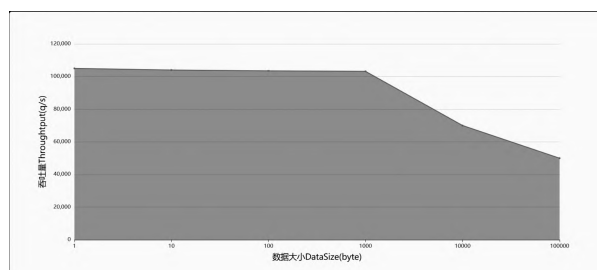


图2 Redis 吞吐量与数据包大小关系图

对于大多数的数据请求,Redis 的处理响应效率都会保持在较高的水平。Redis 为什么能达到那么快的性能,其主要原因如下。

1.1 纯内存的 KV 操作

Redis 是一种非关系型保存在内存中的数据库^[4]。计算机的内存速度相对较快,而 Redis 的绝大部分请求是在内存中处理的。Redis 内部普遍采用 HashMap 的数据结构,而 HashMap 数据结构

的优势是查找和插入操作的时间复杂度都是 $O(1)$,所以 Redis 能够获得很高的响应速度^[5](见表 1)。

表1 不同数据结构的时间复杂度

数据结构	操作	时间复杂度
List	Insert	$O(N)$
List	Select	$O(1)$
Set	Insert	$O(1)$
Set	Select	$O(1)$
HashMap	Insert	$O(1)$
HashMap	Select	$O(1)$

1.2 单线程操作实现

Redis 的使用并不受计算机 CPU 的限制,而是受计算机网络和内存的限制^[6]。Redis 官网也提到 Redis 在使用的时候,使用单线程就已经能够获取 Redis 足够使用的 CPU 资源,主要的瓶颈还是在于计算机的内存,如果使用多线程就会带来多余的资源占用。

1.2.1 上下文切换

计算机程序中的上下文其实就是某种多线程执行状态下 CPU 中寄存器和程序计数器状态。主要的作用就是存放没有被分配到资源的线程。在多线程操作的时候,每个线程同步执行,而 CPU 资源只有一个,不是每一个线程都能够直接获取到 CPU 资源的^[7],某一时刻,必然存在运行、挂起和阻塞状态下的线程。之所以能够看到计算机上能够运行很多的程序,是因为多线程在执行的过程中 CPU 不断对多线程的切换。每个线程都是按照时间片原则来执行的。因此总有线程获取到资源,也总有线程需要等待获取资源,这个时候,等待获取资源的线程就需要被挂起,其现场需要被保存,也就是所说的寄存。这个时候上下文就产生了,当上下文再次被唤起得到资源的时候,就是所说的上下文的切换。单线程则不需要上下文切换,也就减少了这部分的开销。

1.2.2 资源竞争

多线程涉及到资源竞争,而往往线程所需要的资源比计算机实际能提供的资源要多,所以这就涉及到资源的有效分配,而当前没被分配到资源的线程就要等待全部资源的满足才能运行。CPU 对上下文的切换实质是一种资源分配,许多线程都在对资源进行请求,但是在切换之前,到

底切换到哪一个上下文,就是资源竞争的开始。如果资源分配不合理还会出现死锁的风险。Redis 中由于是单线程的,所以所有的操作都不会涉及到资源的竞争,这就减少了等待资源的时间。

1.2.3 加锁的消耗

对于多线程的情况,不能回避的就是锁的问题。这就涉及到多线程之间的并发和同步的问题,多线程操作出现并发,有可能导致数据不一致,或者操作达不到预期的效果。为了实现线程同步,这个时候就需要加锁来解决这些问题。当线程很多的时候,就需要不断的加锁,释放锁。这就会无形中增加了资源的消耗。Redis 的单线程则不会有多线程的并发和同步的问题也就不存在锁的情况,所以资源消耗少,效率高。

1.3 异步非阻塞的 I/O

I/O 操作的阻塞是指当用户线程发出 I/O 请求之后,计算机内核会去查看数据是否准备就绪,如果没有就绪就会等待数据就绪,而用户线程就会处于阻塞状态,用户线程交出 CPU 资源。当数据就绪之后,内核会将数据拷贝到用户线程,返回结果给用户线程,用户线程才解除阻塞状态。Redis 采用的多路复用其实是在单个线程中通过记录跟踪每一个 I/O 流的状态来管理多个 I/O 流。Epoll, poll, select 都是 I/O 多路复用的具体的实现。Epoll 性能比其他几个要好。Redis 中的 I/O 多路复用的所有功能是通过包装常见的 Epoll, poll, select 和 kqueue 这些 I/O 多路复用函数库来实现的。

2 Redis 性能优化策略

Redis 是性能比较好的缓存技术,但是在使用中如果不适当的使用也会限制 Redis 的高效率,不能很好的发挥 Redis 的性能优势。所以,针对不同的使用场景如何最大限度地提高 Redis 的使用效率呢? 以下将从几个方面分别进行分析,并给出相应的解决方案。

2.1 设置键值的过期时间

在实际使用中,当插入键值对的时候,有一个参数可以设置键值对的过期时间,当到达过期时间则自动清除该键值。可以根据业务的实际使用情况,对键值对设置合理的过期时间,一方面可以及时的清除过期的 key 值,减少对 Redis 内存空间的占用,使得更多有用的数据从计算机内存调入 Redis 内存; 另一方面也能在查询数据的时候避免过多的无用键值对堆积,导致 Redis 中

页面的命中时间增加,同时也避免了频繁的触发 Redis 内存的页面淘汰策略以及减少计算机物理内存的页面置换时间。这也是 Redis 在设计过期时间的初衷时极力推荐的一种做法。减少了数据的冗余,显然是能够很大的提高 Redis 的使用性能的。如图 3 所示。

```

管理员: C:\Windows\system32\cmd.exe - redis-cli.exe -h 127.0.0.1 -p 6379
C:\Users\Administrator>K:
K:\>cd K:\Redis-x64-5.0.14
K:\Redis-x64-5.0.14>redis-cli.exe -h 127.0.0.1 -p 6379
127.0.0.1:6379> set a1 xyz
OK
127.0.0.1:6379> set a2 xyz
OK
127.0.0.1:6379> set a3 xyz
OK
127.0.0.1:6379> set a4 xyz
OK
127.0.0.1:6379> expire a1 1000
(integer) 1
127.0.0.1:6379> pexpire a2 10000
(integer) 1
127.0.0.1:6379> ttl a1
(integer) 986
127.0.0.1:6379> ttl a2
(integer) 3
127.0.0.1:6379> expireat a3 1612387653
(integer) 1
127.0.0.1:6379> ttl a3
(integer) -2
127.0.0.1:6379> pexpireat a4 161238765786
(integer) 1
127.0.0.1:6379> ttl a4
(integer) -2
  
```

图 3 Redis 设置键值对过期时间

虽然可以设置每个键的过期时间,但是同样也要注意避免发生缓存雪崩问题。缓存雪崩是指设置缓存时很多采用了相近的过期时间,当缓存中大批量数据都达到这一过期时间时,会导致请求量全部请求到服务端数据库,最终导致数据库瞬时压力过大而崩溃。造成数据库宕机,这就是缓存雪崩现象。如图 4 所示。

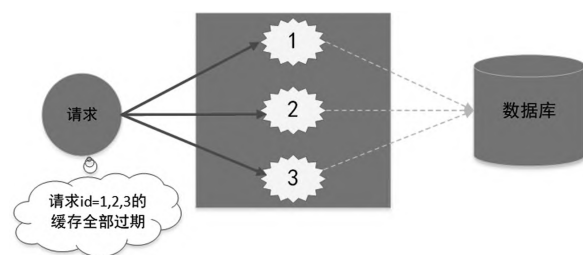


图 4 缓存雪崩示意图

解决此问题的方法就是预防大量的 key 在同一时刻过期,最简单的方案就是在原有失效时间的基础上增加一个随机时间,例如 1~5 分钟,这样每个缓存过期时间的重复率就会降低,从而减少缓存雪崩的发生。或者采用双缓存机制,缓存 A 和 B,比如 A 的失效时间是 20 min, B 不失效。比如从 A 中没读到,就去 B 中读,然后异步起一个线程同步到 A。还可以使用异步重建缓存,在这种

方案下构建缓存采取异步策略,会从线程池中获取线程来异步构建缓存,从而不会让所有的请求直接到达存储层,该方案中每个 key 维护逻辑超时时间,当逻辑超时时间小于当前时间时,则说明当前缓存已经失效,应当进行缓存更新,否则说明当前缓存未失效,直接返回缓存中的值。如在 Redis 中将 key 的过期时间设置为 100 min,在对应的 Value 中设置逻辑过期时间为 50 min。这样当 key 到了 50 min 的逻辑过期时间,就可以异步更新这个 key 的缓存,但是在更新缓存的这段时间内,旧的缓存依然可用。这种异步重建缓存的方式可以有效避免大量的 key 同时失效。还有一种最简单的方式就是不设置缓存的过期时间,这样就不会出现大量缓存同时失效的问题,但随之而来的是 Redis 需要更多的存储空间。

2.2 限制 Redis 的内存占用大小

Redis 通过设置 maxmemory 来设置 Redis 的最大内存。然而在 64 位操作系统中,因为 maxmemory 配置项是被注释掉的,所以 Redis 的内存大小没有限制。这样就会导致在 Redis 内存不足时会使用磁盘作为其虚拟内存,而当操作系统将 Redis 所用的内存分配至磁盘时,将会阻塞 Redis 进程,导致 Redis 出现延迟,从而影响 Redis 的整体性能,因此要限制 Redis 的内存大小为一个固定的值,并且该值不能大于服务器的内存。当 Redis 的运行达到此值时会触发内部的淘汰策略,从而将内存回收。

如图 5 所示,从实验图中的趋势可以看出在实际使用中,Redis 内存在一开始是随着占据内存的增加命中率逐步提高的,但是当达到一定的量级后,其命中率并不会随着内存的继续增加而增加,反而会降低。刚开始 Redis 内存的增加会使得一些操作都转移到 Redis 中完成,许多所需的数据都在 Redis 中命中,无需访问数据库。而随着 Redis 内存的逐步增加及数据的增多必然导致频

繁的进行与内存的页面置换操作,数据量的增多也会导致检索时间的增加。频繁的页面置换往往也会导致一些所需的页面不在 Redis 内存中而不得不重新到内存中去查找所需的数据,导致命中率逐步降低。所以从图 5 的趋势中建议 Redis 内存大小设置在物理内存的 1/2 到 1/4 为最佳 Redis 内存大小。

如表 2 所示,页面淘汰算法主要有 LRU 和 LFU, LRU 即最近最少未使用,是淘汰最长时间没有被使用的页面,其侧重的是最近被访问的数据不删除,而 LFU 即最不经常使用淘汰算法,淘汰一段时间内,使用次数最少的页面,是保证访问频率最高的数据不被删除,但是 LFU 有一个缺点,就是如果在某一个时段,某个 key 的访问频率非常高,那么该 key 就会变为热点数据,但是实际上该 key 在其他时间节点不会被用到,那么会造成这些数据不会被删除。所以可通过 LRU 替换算法来限制内存的使用。

表 2 Redis 中的 8 种淘汰策略

淘汰策略	说 明
noeviction	不淘汰任何数据,当内存不足时,新增操作会报错,默认淘汰策略
allkeys-lru	淘汰整个键值对中最近未使用的数据
allkeys-random	随机淘汰任意值
volatile-lru	淘汰设置了过期时间 key 中最久 key
volatile-random	随机淘汰设置了过期时间的 key
volatile-ttl	优先淘汰最早过期的 key
volatile-lfu	淘汰设置过期时间 key 使用最少 key
allkeys-lfu	淘汰所有 key 中的最少被使用的 key

2.3 Redis 内存优化设计

要知道 Redis 中数据的占用内存大小,需要对 Redis 的内存模型有个全面的了解。假设 Redis 中有 m 个键值对,且 Redis 键值都不是整数类型,则占用的空间可以分为两部分,一部分是 m 个 dictEntry 占据的字符串,每个 dictEntry 占据的空间包括:一个 key,一个 dictEntry 结构、一个 Object。一个 value,另一部分是键值对所需要的 bucket 空间。其空间即为两部分之和。则由内存模型出发可对其做如下优化(如图 6)。

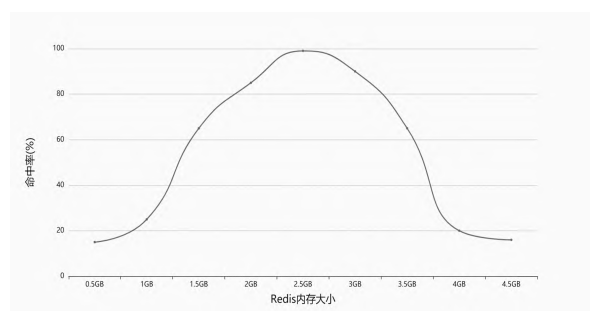


图 5 Redis 内存大小与命中率的关系图

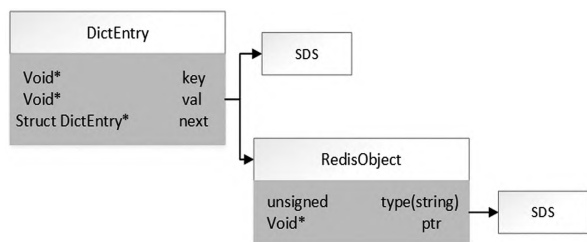


图6 Redis内存存储结构

2.3.1 使用内存分布器 jemalloc 特性优化

由于简单动态字符串 SDS 等于字符串长度加 4, jemalloc 分配的空间是不连续的, 因此 KV 字符串变化一个字节, 可能会引起占用内存的很大变化, 在设计时可以利用这一点。例如如果 key 的长度是 13, 则 SDS 为 17 个字节, 那么 jemalloc 就会分配 32 个字节; 如果将 key 的长度缩减为 12 个字节, 则 SDS 为 16 个字节, jemalloc 会分配 16 个字节, 这样每个 key 所占用的空间就可以缩小一半。

2.3.2 尽量使用整形/长整形

使用字符串会造成空间的浪费, 如果使用整型或长整型, Redis 会使用 int 类型(8 个字节)存储来替代字符串类型, 使得其长度都是 8 的整数倍, 这样计算机在分配空间的时候可以避免一些小的碎片的产生, 这样可以节省更多资源空间, 让更多的数据存储到 Redis 缓存中, 减少了数据库的访问, 从而提升 Redis 的使用效率。因此在可以使用整型或长整型代替字符串的场景下, 我们应该尽量使用整型或长整型。

2.3.3 共享对象

利用已经建立的共享对象, 则可以大幅度减少 Redis 中对象的创建和销毁, 同时也相应的减少了 RedisObject 的创建, 对象的创建和销毁需要时间并且耗费计算机资源, 所以使用共享对象从而节省空间。

2.3.4 缩短键值对的存储长度

键值对的长度同样也影响着 Redis 的性能, 键值对的长度是和性能成反比的, 所以在使用中应尽量缩小键值对。由实验得出如下结论(数据量为 100 万)。由表 3 中可知, 在数据量一定 key 保持不变的情况下, value 值越大则操作效率越低, 因为 Redis 对于同一种数据类型会使用不同的内部编码进行存储, 比如字符串的内部编码就有三种: int、embstr、raw, 这是因为 Redis 是想通过不同的编码实现效率和空间的平衡, 然而数据量

表3 Redis 键值对长度效率影响

key 大小 (/byte)	value 大小 (/byte)	string:set 平均耗时(/μs)	string:hset 平均耗时(/μs)
20	512	1.13	10.28
20	200	0.74	8.08
20	100	0.65	7.92
20	50	0.59	6.74
20	20	0.55	6.60
20	5	0.53	6.53

越大使用的内部编码就越复杂, 而越复杂的内部编码存储的性能就越低。同时内容越大, 需要的持久化、挂起的时间就越长, 网络上传输的内容多, 时间就越长, 占用的内存就会多, 以至于频繁的触发内存淘汰机制。所以在保证业务完整的同时, 尽量缩短键值对的存储长度, 也可以在需要的时候对键值对进行压缩存储或者序列化操作。

2.4 缓存穿透优化

缓存穿透是指查询一个在缓存和数据库中都不存在数据, 这样当缓存层里面没有时, 就会去访问后面的存储层了(如图 7)。

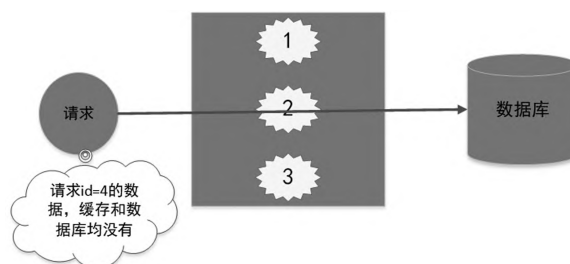


图7 缓存穿透示意图

如果有大量的这种恶意请求过来, 都会转向后面的存储层。显然存储层是扛不住这样的压力。这样缓存就失去了保护后面存储的意义了。解决方法如下(如图 8)。

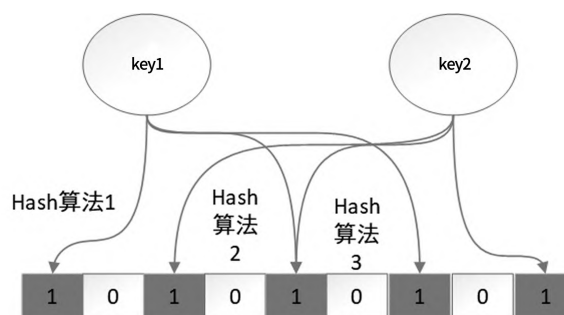


图8 布隆过滤器 Hash 映射方式

2.4.1 布隆过滤器优化

其原理是对于不存在的数据布隆过滤器一般都能够过滤掉,不让请求再往后端发送。当布隆过滤器说某个值存在时,这个值可能不存在;但是它说不存在时,那就肯定不存在。布隆过滤器是一个大型的位数组和几个不一样的无偏 Hash 函数。所谓无偏就是能够把元素的 Hash 值算得比较均匀。向布隆过滤器中添加 key 时,会使用多个 Hash 函数对 key 进行 Hash 运算分别算得一个整数索引值然后对位数组长度进行取模运算得到一个位置,每个 Hash 函数都会算得一个不同的位置。再把位数组的这几个位置都置为 1 就完成了添加操作。向布隆过滤器询问 key 是否存在时跟添加一样,也会把 Hash 的几个位置都算出来,看看位数组中这几个位置是否都为 1,只要有一个位为 0,那么说明布隆过滤器中这个 key 肯定不存在。但是都是 1,这并不能说明这个 key 就一定存在,只是极有可能存在,因为这些位被置为 1 可能是因为其它的 key 存在所致。这种方式能很好地提升缓存的利用率^[9]。

表 4 不同 Hash 函数个数对碰撞误判率影响

空间大小	集合大小	Hash=1	Hash=2	Hash=3	Hash=4
1000	100	5	1	0	0
10000	1000	42	14	9	2
100000	10000	524	114	53	28
1000000	100000	4498	1117	451	262

如表 4 所示,从实验数据中可以得出当只有 1 个 Hash 函数的时候,误判率很高,但是当有 4 个 Hash 函数的时候已经缩小接近 20 多倍,可以动态根据业务需求所允许的识别率来调整 Hash 函数的个数,找出最适合的 Hash 函数个数,当然 Hash 函数也不是越多越好,太多的 Hash 函数所带来的空间效率和查询效率也会有所降低。

2.4.2 接口层增加校验

在接口层增加鉴权校验,可以设定请求的合格条件,当外在请求不能满足条件时则禁止其访问数据库服务器和缓存,避免了无效数据的频繁恶意攻击。

2.4.3 设置空数据值为 null

当从缓存取不到的数据,在数据库中也没有

取到,这时也可以将 key-value 对写为 key-null,直接返回空值。缓存有效时间可以设置短点,如 30 秒。这样就能避免短时间内的大量无效请求负担。

2.5 缓存击穿优化

缓存击穿是指缓存中没有但数据库中有数据,当一个 key 短期内频繁访问,在不停的进行请求,大并发集中对这一个点进行访问;当这个 key 在失效的瞬间,持续的大并发就冲破缓存,直接请求数据库,就像在一个屏障上凿开了一个洞。针对这种问题一方面可以设置热点数据永不过期解决,另一种可以通过添加互斥锁代码实现对数据的访问(如图 9)。

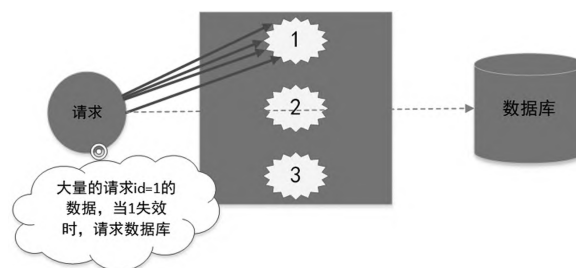


图 9 缓存击穿示意图

互斥锁代码如下:

```

Public static String getData(String key)
throws InterruptedException{
//从缓存中读取数据
String data = getDataFromRedis(key);
if(data==null){//缓存中不存在数据
//去获取锁,获取成功,去数据库取数据
if(reenLock.tryLock()){
//从数据库中获取数据
data=getDataFromSql(key);
if(data!=null){//更新缓存数据
setDataToRedis(key,data);}
reenLock.unlock();//释放锁
}else{//获取锁失败
//暂停 100ms 后再重新获取数据
Thread.sleep(100);
data=getData(key);}
return data;
}

```

如果缓存中有数据,则直接返回,如果没有,则第一个进入的线程先去查询数据库,并加上锁,其他线程则等待,这样就能防止去数据库查重复数

据、重复更新缓存了。

2.6 使用 Redis 管道技术

管道技术是客户端提供的一种批处理技术, Redis 是支持管道的,用于一次处理多个 Redis 命令,Redis 管道技术在服务端未响应时,客户端可以继续向服务端发送请求,并最终一次性读取所有服务端的响应。从而提高整个交互的性能(如图 10)。

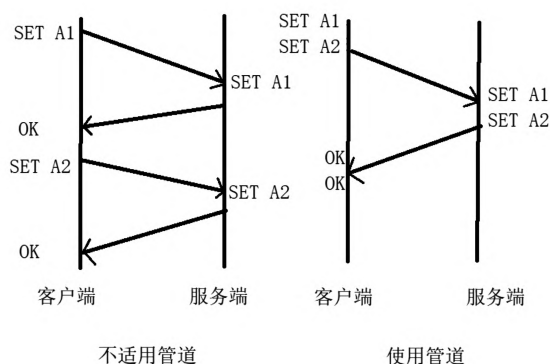


图 10 有无管道 Redis 命令处理图

使用 Java 代码来测试一下 Pipeline 和普通操作的性能对比, Pipeline 测试代码如下:

```
Public class PipeExample{
Public static void main(String[]
args){Jedis jedis=new
Jedis("127.0.0.1",6379);
//记录执行开始的时间
Long bTime=System.curentTimeMillis();
//获取 PipeLine 对象
PipeLine pl=jedis.PipeLined();
//设置多个 Redis 命令
for(int i=0;i<100;i++){
    pl.set("key"+i,"val"+i);
    pl.del("key"+i);}//执行命令
pl.sync();//记录执行结束时间
Long eTime=System.curentTimeMillis();
System.out.println("执行耗时:"
+(eTime-bTime)+"毫秒");}}
以上程序执行结果为:执行耗时:302 毫秒
普通的操作代码如下:
Public class PipeExamplePt{
Public static void main(String[]
args){Jedis jedis=new
Jedis("127.0.0.1",6379);
//记录执行开始的时间
Long bTime=System.curentTimeMillis();
```

```
for(int i=0;i<100;i++){
    jedis.set("key"+i,"val"+i);
    jedis.del("key"+i);}
//记录执行结束时间
Long eTime=System.curentTimeMillis();
System.out.println("执行耗时:"
+(eTime-bTime)+"毫秒");}}
以上程序执行结果:执行耗时:18003 ms。
```

从以上结果可以看出,管道的执行时间是 302 ms,而普通命令执行时间是 18003 ms,管道技术要比普通的执行大约快了 58 倍。很明显利用管道技术可以大幅度提高 Redis 的使用性能。

2.7 使用 lazy free 特性

2.7.1 什么是 lazy free

Lazy free 是 Redis 最新的技术,它可以理解为惰性删除或延迟删除,意思就是在删除的时候提供异步删除的功能,它把删除 key 的操作放在 BIO 单独的子线程处理中,以减少删除操作对 Redis 主线程的阻塞,同时可以有效的避免删除大的 key 时带来的性能和可用性问题。

2.7.2 使用 lazy free 优势

Redis 除少量的 BIO 任务外基本上是单线程程序,当运行一个耗时较大的请求时,会导致所有请求排队等待,Redis 不能响应其他请求,从而引起性能问题,甚至集群发生故障切换。而 Redis 删除大的集合键时,就属于这类比较耗时的请求。通过测试来看,删除一个 100 万个元素的集合键,耗时约 1000 ms 左右(如表 5)。

表 5 Redis 不同 key 类型大集合删除效率

key 类型	Item 数量(万)	耗时
Hash	100	999.2 ms
List	100	1001.3 ms
Set	100	1000.2 μ s
Sorted Set	100	999.6 μ s

以下测试,删除一个 100 万个字段的 Hash 键,耗时 1360 ms;处理此 DEL 请求期间,其他请求完全被阻塞(如图 11、图 12)。

引入 lazy free 后,这类主动或被动的删除 big key 时,和一个 O(1)指令的耗时一样,亚毫秒级返回;把真正释放 Redis 元素耗时动作交由 BIO 后台任务执行。示例如下:使用 UNLINK 命令删除

```
1 删除一个100万字段的hash键
2 127.0.0.1:6379> HLEN hlazykey
3 (integer) 1000000
4 127.0.0.1:6379> del hlazykey
5 (integer) 1
6 (1.36s)
7 127.0.0.1:6379> SLOWLOG get
8 1) 1) (integer) 0
9 2) (integer) 1501314385
10 3) (integer) 1360908
11 4) 1) "del"
12 2) "hlazykey"
13 5) "127.0.0.1:35595"
6) ""
```

图 11 Redis 大 hash 键删除测试

```
1 127.0.0.1:7000> LLEN mylist
2 (integer) 2000000
3 127.0.0.1:7000> UNLINK mylist
4 (integer) 1
5 127.0.0.1:7000> SLOWLOG get
6 1) 1) (integer) 1
7 2) (integer) 1505465188
8 3) (integer) 30
9 4) 1) "UNLINK"
10 2) "mylist"
11 5) "127.0.0.1:17015"
6) ""
```

图 12 Redis 主动删除大键测试

一个大键 mylist,它包含 200 万个元素,但用时只有 0.03ms。显然比上面没有引入 lazy free 技术删除 100 万个元素需要 1360ms 性能有了很大的提高。

2.8 其他优化策略

2.8.1 客户端优化

在客户端还需要尽量使用 Redis 连接池,而不是频繁的建立、销毁 Redis 连接。这样就可以减少网络传输次数和减少非必要的调用命令。

2.8.2 禁用长耗时的查询命令

Redis 中大多数的读写命令都能在时间复杂度为 $O(1)$ 的情况下完成,因此效率极高,但也不可避免的会遇到复杂度为 $O(N)$ 的读写操作,这时就需要注意查询速度慢的问题,因为 Redis 是单线程的,查询只有一个线程操作,对于耗时的指令会出现 Redis 的延迟。可以进行以下改进(如表 6)。

2.8.3 使用 slowlog 优化耗时的命令

这种方法可以帮助找出哪些命令是比较耗时的指令,从而可以有针对性地进行单独优化从而提高 Redis 的性能。Redis 中存在着许多读写命令,他们执行的时间不一,那么必然存在一些耗费时间比较多的命令,如果耗费时间比较多的命令大量存在则势必会降低 Redis 的效率,所以需要对这些高耗时的命令进行查询检测。使用 showlog 功能就能查询出最耗时的 Redis 命令,进而进行优化来提高 Redis 的速度。

表 6 Redis 长耗时查询命令改进

序号	改进措施
1	禁止使用 key * 命令
2	避免一次查询所有成员,要使用 scan 命令进行分批的、游标式的遍历
3	通过机制严格控制 Hset/Set/Sorted Set 等数据结构的大小
4	将排序、并集、交集等操作放在客户端执行,以减少 Redis 服务器的运行压力
5	删除大的数据时,可能会需要很长的时间,建议用异步删除的方式 unlink,它启动一个新的线程来删除目标数据,而不是阻塞 Redis 主线程。

2.8.4 使用分布式架构来增加读写速度

Redis 的分布式架构有主从同步、哨兵模式、Redis Cluster 集群,主从同步可以将写操作放在 master 上处理,可以将读操作放在 slave 上处理,因此极大的提高了 Redis 的处理性能;而 Sentinel 是对主从同步的一个升级,解决了单点故障以及故障迁移的问题;而 RedisCluster 集群,在 Redis3.0 推出,解决了 Redis 的扩展问题,并且将读写操作使用 hash 分布到了不同的 Redis 服务器上,极大的提高了 Redis 命令的操作性能。选择上首选 Redis Cluster 集群的实现方案。

2.8.5 使用物理机而非虚拟机

在虚拟机中运行 Redis 服务器,因为和物理机共享一个物理端口,并且一台物理机可以有多个虚拟机在运行,因此在内存占用上和网络延迟方面会有很糟糕的表现,可以使用 ./redis-cli --intrinsic-latency100 命令来查看延迟时间,如果对 Redis 性能要求很高的话,应尽可能在物理机上直接部署 Redis 服务器。

3 结语

在当前 Web 信息系统的发展下,面对大数据量、高并发、高响应比的需求,Redis 都很好的解决了以上问题,同时 Redis 这些年也在根据用户的需求不断的更新功能和性能优化,使得 Redis 能更加灵活的使用在多个业务场景中,同时也需要我们在使用中认真的思考,在性能上选择最适合自己业务系统的功能,合理的利用 Redis 的多功能,提升 Redis 的业务体验。

(下转第 96 页)

- [21] 吕飘飘,谢丹丹,张朝晖.碳点基荧光印迹聚合物的选择性检测牛血红蛋白[J].分析化学,2018(6):917-924.
- [22] 谭丽菊,付海璐,史恬,等.CdTe 量子点分子印迹传感器对水中磺胺类药物残留的检测[J].光学精密工程,2018(9):2254-2261.
- [23] LI A, JIA J, FAN Y, et al. Furfural and organic acid targeted carbon dot sensor array for the accurate identification of Chinese baijiu[J]. Journal of food science, 2021(7):2924-2938.
- [24] 郭艳玲,庞志远,安丽娟.氧氟沙星磁性分子印迹聚合物的制备及性能研究[J].食品科技,2015(7):347-350.
- [25] 刘雨星,管桂芹,何锡文,等.核壳式碳纳米管-茶多酚分子印迹聚合物的制备及应用[J].分析化学,2013(2):161-166.
- [26] 赵晓磊,王俊平.碳量子点分子印迹荧光传感器检测多巴胺[J].分析实验室,2019(11):1282-1288.
- [27] DING H, JIAO H, SHI X, et al. Molecularly imprinted optosensing sensor for highly selective and sensitive recognition of sulfadiazine in seawater and shrimp samples[J]. Sensors and Actuators B Chemical, 2017, 246: 510-517.
- [28] 丘秀珍,李勇莹,彭翠红.分子印迹荧光传感器的研制及其对呋喃妥因的特异性识别[J].分析测试学报,2018(3):331-335.
- [29] 刘巧玲,任博荣,刘睿蓉,等.N-掺杂的荧光碳点比率识别 Ag⁺的性能[J].应用化学,2021(11):1512-1520.
- [30] 何雨,王硕,王俊平.碳点分子印迹传感器对水产品 and 动物产品中己烯雌酚残留的检测[J].分析实验室,2020(7):755-759.

责任编辑:刘 运

(上接第 87 页)

参考文献:

- [1] 秦琛,艾青.基于 Redis 技术的页面缓存研究[J].科技风,2021(12):74-76.
- [2] 张俊.基于 Redis 实现关系型数据库内存化研究[D].成都:四川师范大学,2021.
- [3] 伍华锋,王小刚,候劲,等.Redis 集群系统的优化研究与实现[J].四川轻化工大学学报(自然科学版),2020(6):39-45.
- [4] 胡雅倩.基于 Redis 限制用户访问频率的研究[J].网络新媒体技术,2020(6):21-25+50.
- [5] 王晓燕.Redis 基本数据类型的使用[J].办公自动化,2020(23):48-49+58.
- [6] 唐天兵,柳青.基于 Redis 的热点 Key 统计方案研究与实现[J].电脑知识与技术,2020(31):248-250.
- [7] 张审问,吴海燕,李星,等.基于 Redis 的水务云系统缓存替换策略设计[J].水利信息化,2020(4):16-21.
- [8] 贾小云,杜晓旭.多维布隆算法在 Redis 指纹自动过期中的应用[J].计算机应用与软件,2020(8):33-38.
- [9] 展维维.基于 Redis 的分布式存储系统设计与应用[D].南京:南京大学,2020.
- [10] 张杰,刘凯,周立军.采用 Redis 高并发应用系统设计与实现方法[J].计算机与数字工程,2020(5):1222-1226.
- [11] 宋云奎,吴文鹏,赵磊,等.基于 Redis 的分布式数据存储方法[J].计算机产品与流通,2020(8):106.

责任编辑:徐 健